



Comprehensive Project Report:

FraudShield Pro

Executive Summary

Credit card fraud poses a multi-billion-dollar challenge to global financial institutions. Detecting fraudulent transactions in real-time requires algorithms capable of operating under extreme class imbalance without swamping security operations centers (SOCs) with false alarms.

FraudShield Pro addresses this challenge through an end-to-end Machine Learning pipeline paired with an executive interactive dashboard. The system achieves a **ROC-AUC score of 0.9836** and an **83% Fraud Recall rate** while maintaining **78% Precision**, offering both high sensitivity and low false-positive rates.

1. Project Objectives

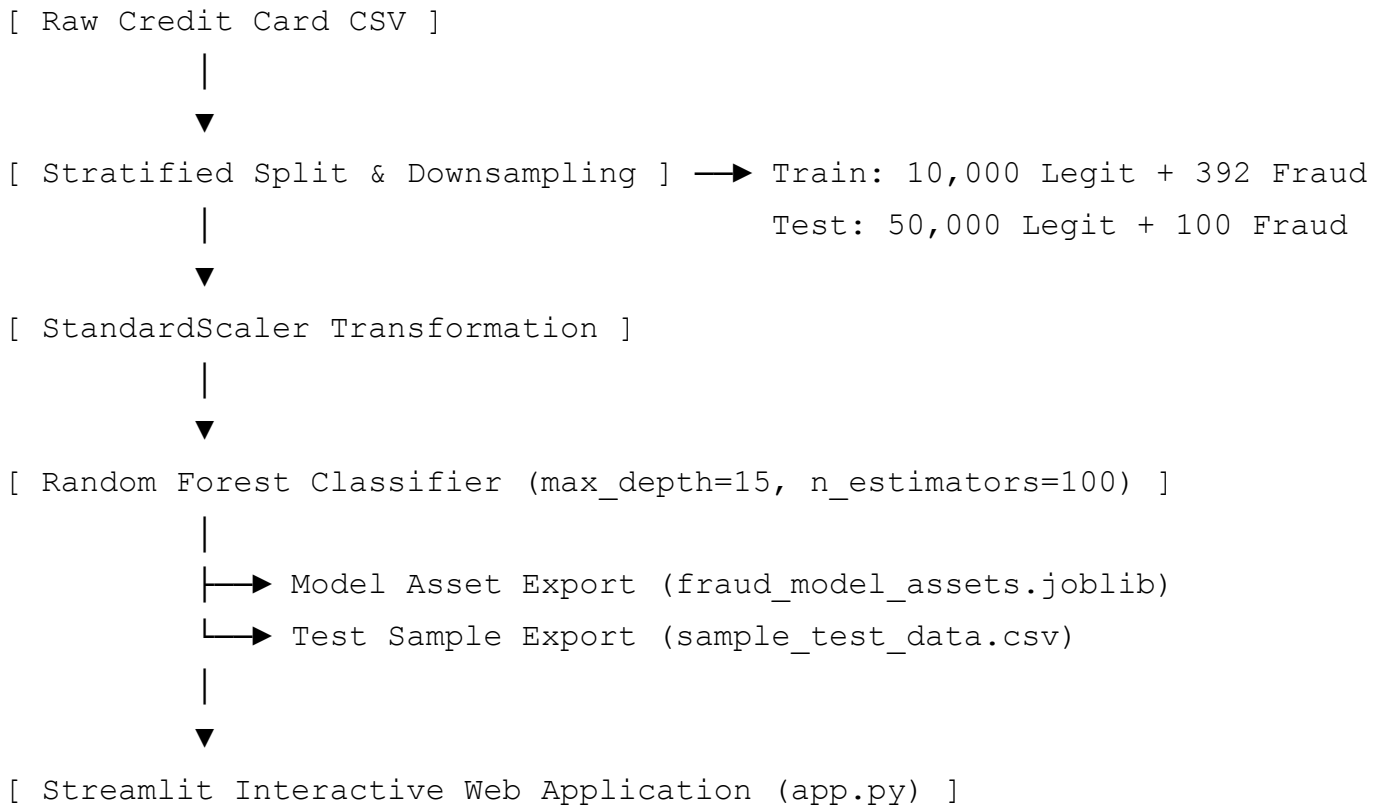
1. **Accurate Anomaly Detection:** Build a machine learning classifier capable of flagging fraud in highly imbalanced dataset environments.
 2. **Statistical Explainability:** Translate complex PCA features into clear Z-Score distance metrics to explain *why* a specific transaction was flagged.
 3. **Interactive Visual Dashboard:** Provide financial analysts with an intuitive web application for model evaluation, real-time testing, and exploratory data analysis.
-

2. Dataset Overview

- **Source:** European Cardholders Credit Card Fraud Dataset (Kaggle).
- **Total Transactions:** 284,807
- **Legitimate Transactions (Class 0):** 284,315 (99.83%)
- **Fraudulent Transactions (Class 1):** 492 (0.17%)
- **Features:**
 - **Time** : Seconds elapsed between each transaction and the first transaction in the dataset.
 - **Amount** : Transaction dollar amount.
 - **v1** through **v28** : Principal Component Analysis (PCA) transformed numerical features to preserve confidentiality.

- `Class` : Target variable (`0` = Legitimate, `1` = Fraudulent).
-

3. Methodology & System Architecture



3.1 Data Preparation & Imbalance Strategy

To handle severe class skewness (0.17% fraud rate):

- **Downsampling:** Downsampled majority class (Legitimate) to 10,000 instances while retaining 392 Fraud instances for model training.
- **Realistic Testing Environment:** Evaluated on an un-downsampled test set of 50,100 transactions (50,000 Legitimate, 100 Fraud) to mirror real-world production conditions.

3.2 Machine Learning Model

- **Algorithm:** `RandomForestClassifier(n_estimators=100, max_depth=15, random_state=42, n_jobs=-1)`
 - **Preprocessing:** `StandardScaler` applied to align variance scales across `Time`, `Amount`, and `V1-V28`.
-

4. Experimental Results & Model Performance

4.1 Evaluation Metrics

Metric	Score	Explanation
Accuracy	99.92%	Total ratio of correctly classified transactions.
Precision (Fraud)	78.00%	78% of transactions flagged as fraud are true fraud.
Recall (Fraud)	83.00%	83% of all actual fraud cases were successfully caught.
F1-Score (Fraud)	0.81	Harmonic mean of precision and recall.
ROC-AUC	0.9836	Area under ROC Curve demonstrating high discrimination power.

4.2 Confusion Matrix (Test Set: 50,100 Cases)

Actual \ Predicted	Predicted Legitimate	Predicted Fraud
Actual Legitimate	49,977 (True Negative)	23 (False Positive)
Actual Fraud	17 (False Negative)	83 (True Positive)

5. Decision Assessment & Statistical Explainability

To assist fraud investigation teams, FraudShield Pro calculates **Z-score feature deviations** comparing each incoming transaction against legitimate baseline profiles:

$$Z\text{-Score} = \frac{x_{\text{scaled}} - \mu_{\text{legit}}}{\sigma_{\text{legit}} + \epsilon}$$

Features with the highest absolute Z-scores (e.g., **V17**, **V14**, **V12**, **V10**) are highlighted as primary risk drivers.

6. Software Architecture & Implementation

- train_model.py**: Handles dataset loading, train/test splitting, scaler fitting, model training, metric evaluation, asset serialization via **joblib**, and test sample export.
- app.py**: Streamlit application rendering custom CSS themes, 3-tab layout (Predictor, EDA, Performance), interactive Plotly visual charts, and real-time transaction scoring.

7. Conclusions & Future Work

Key Achievements

- Built an effective ML classifier with **0.9836 ROC-AUC**.
- Created an intuitive, dark-themed Streamlit application for real-time analysis.
- Provided statistical anomaly explanations for decision support.

Future Recommendations

1. **XGBoost / LightGBM Benchmarking:** Test gradient boosted decision trees for execution speed improvements.
2. **Deep Learning Integration:** Experiment with Autoencoders for unsupervised anomaly detection.
3. **API Deployment:** Wrap the inference engine in a FastAPI containerized REST endpoint.